

Vehicle Rental Portal

Java Class Documentation & Relationship Reference

13 Java source files · OOP · Interfaces · Threads · JDBC · Swing GUI
Project: C:\Projects\VehicleRental\src

Table of Contents

1. Architecture Overview & Class Relationship Diagram
2. Vehicle.java — Abstract Base Class
3. Car.java — Concrete Subclass
4. Bike.java — Concrete Subclass
5. PremiumVehicle.java — Multiple-Interface Class
6. GPS.java — Interface
7. Insured.java — Interface
8. InvalidDaysException.java — Custom Exception
9. DBConnection.java — Database Utility
10. DatabaseManager.java — JDBC Operations
11. ReceiptGenerator.java — File I/O Utility
12. SystemClockThread.java — Multithreading
13. VehicleRentalGUI.java — Swing GUI
14. Main.java — Application Entry Point
15. Complete Class Relationship Summary

1. Architecture Overview

The VehicleRental project is a Java Swing desktop application that demonstrates core Object-Oriented Programming concepts. It is organized into four logical layers:

Layer	Classes	Responsibility
Presentation	VehicleRentalGUI, Main	Swing-based GUI; application entry point
Domain / OOP	Vehicle, Car, Bike, PremiumVehicle, GPSAssisted	Class hierarchy, interfaces, polymorphism
Services	SystemClockThread, ReceiptGenerator	Background thread, file output
Data Access	DBConnection, DatabaseManager	SQLite JDBC connection and CRUD operations
Exceptions	InvalidDaysException	Business-rule validation

Inheritance Hierarchy (ASCII)

```
java.lang.Object ■■ Vehicle (abstract) ■■ Car ■ ■■ PremiumVehicle implements Insured, GPS
■■ Bike java.lang.Exception ■■ InvalidDaysException javax.swing.JFrame ■■ VehicleRentalGUI
java.lang.Thread (via Runnable) ■■ SystemClockThread implements Runnable
```

2. Vehicle.java

Kind: Abstract Class **Lab topic:** LAB 2 – Inheritance & Polymorphism | LAB 6 – Abstraction

Purpose: The root of the vehicle type hierarchy. Defines the shared state and contract that every concrete vehicle must honour. Because it is *abstract*, no Vehicle instance can be created directly — only Car, Bike, or PremiumVehicle objects exist at runtime.

Fields

Field / Member	Type	Description
vehicleId	protected String	Unique identifier for the vehicle (e.g. CAR001)
vehicleName	protected String	Human-readable model name (e.g. Maruti Swift)
rentPerDay	protected double	Base daily rental rate in Rupees

Methods

Method	Parameters	Returns	Description
Vehicle(id, name, rate)	String, String, double	—	Constructor called by child classes via super()
calculateBaseRent(days)	int days	double	Abstract — each subclass computes its own rate
displayDetails()	—	String	Abstract — returns formatted vehicle info
getVehicleId()	—	String	Getter for vehicleId
getVehicleName()	—	String	Getter for vehicleName
getRentPerDay()	—	double	Getter for rentPerDay

Relationships

Class / File	Relationship	Target	Details
Vehicle	is extended by	Car	Car inherits all three protected fields
Vehicle	is extended by	Bike	Bike inherits all three protected fields
Vehicle	is extended by	Car → PremiumVehicle	PremiumVehicle inherits through Car
VehicleRentalGUI	uses (polymorphism)	Vehicle	selectedVehicle field declared as Vehicle; actual type is Car / Bike / PremiumVehicle

3. Car.java

Kind: Concrete Class (extends Vehicle) **Lab topic:** LAB 2 – Inheritance & Polymorphism

Purpose: Represents a standard car available for rent. Adds the numberOfSeats field and overrides both abstract methods from Vehicle. Rent is calculated at the plain daily rate with no modifier.

Fields

Field / Member	Type	Description
numberOfSeats	private int	Seating capacity of this car (inherited fields from Vehicle also apply)

Methods

Method	Parameters	Returns	Description
Car(id, name, rate, seats)	String, String, double, int	—	Calls super(id, name, rate); stores numberOfSeats
calculateBaseRent(days)	int days	double	Returns rentPerDay × days (standard rate, no modifier)
displayDetails()	—	String	Returns Car ID, model, seats, and rent/day as formatted string
getNumberOfSeats()	—	int	Getter for numberOfSeats

Relationships

Class / File	Relationship	Target	Details
Car	extends	Vehicle	Inherits vehicleId, vehicleName, rentPerDay
Car	is extended by	PremiumVehicle	PremiumVehicle builds on Car's constructor via super()
VehicleRentalGUI	instantiates	Car	new Car("CAR001","Maruti Swift", 1500.0, 5) when user select

4. Bike.java

Kind: Concrete Class (extends Vehicle) **Lab topic:** LAB 2 – Inheritance & Polymorphism

Purpose: Represents a two-wheeler (scooter or sports bike). Adds the bikeType field and demonstrates that multiple concrete classes can share the same abstract parent while each providing distinct behaviour. Bikes get a built-in 10 % discount on the listed daily rate.

Fields

Field / Member	Type	Description
bikeType	private String	Category of bike, e.g. Scooter or Sports Bike

Methods

Method	Parameters	Returns	Description
Bike(id, name, rate, type)	String, String, double, String	—	Calls super(id, name, rate); stores bikeType
calculateBaseRent(days)	int days	double	Returns (rentPerDay × days) × 0.90 — 10 % discount baked in
displayDetails()	—	String	Returns Bike ID, model, type, and rent/day as formatted string
getBikeType()	—	String	Getter for bikeType

Relationships

Class / File	Relationship	Target	Details
Bike	extends	Vehicle	Inherits vehicleId, vehicleName, rentPerDay
VehicleRentalGUI	instantiates	Bike	new Bike("BIKE001","Honda Activa", 500.0, "Scooter") when u

5. PremiumVehicle.java

Kind: Concrete Class (extends Car, implements Insured & GPS) **Lab topic:** LAB 3 – Multiple Inheritance via Interfaces

Purpose: The most feature-rich vehicle type. Extends Car (single class inheritance) and simultaneously implements two interfaces (GPS and Insured) to simulate Java's multiple-inheritance capability. Charges a 25 % premium on the base car rate and exposes insurance and GPS details via the interface contracts.

Fields

Field / Member	Type	Description
insuranceProvider	private String	Name of the insurance company (e.g. HDFC Ergo)
gpsDeviceId	private String	Identifier of the fitted GPS device (e.g. GPS-2024-PREM)

Methods

Method	Parameters	Returns	Description
PremiumVehicle(id,name,rate,Seats,insuranceProvider,gpsDeviceId) String	String id, String name, double rate, int Seats, String insuranceProvider, String gpsDeviceId	—	Calls super(id,name,rate,seats) then stores insurance and GPS details
calculateBaseRent(days)	int days	double	Returns super.calculateBaseRent(days) × 1.25 — 25 % surcharge
displayDetails()	—	String	Returns super.displayDetails() appended with premium features
getInsuranceInfo()	—	String	Implements Insured interface; returns provider + coverage type
getGPSInfo()	—	String	Implements GPS interface; returns device ID + tracking status

Relationships

Class / File	Relationship	Target	Details
PremiumVehicle	extends	Car	Inherits numberOfSeats and all Vehicle fields
PremiumVehicle	implements	Insured	Must provide getInsuranceInfo()
PremiumVehicle	implements	GPS	Must provide getGPSInfo()
VehicleRentalGUI	instantiates	PremiumVehicle	Created when user selects 'Premium Car'; cast checked with instanceof

6. GPS.java

Kind: Interface **Lab topic:** LAB 3 – Multiple Inheritance via Interfaces

Purpose: Declares the contract for GPS-capable vehicles. Any class that implements GPS must provide a `getGPSInfo()` method that returns tracking details. Used alongside `Insured` to demonstrate that a single Java class can fulfil multiple interface contracts.

Contract Method

Method	Parameters	Returns	Description
<code>getGPSInfo()</code>	—	String	Returns GPS device ID and real-time tracking status

Relationships

Class / File	Relationship	Target	Details
GPS	is implemented by	PremiumVehicle	PremiumVehicle provides the concrete <code>getGPSInfo()</code> body

7. Insured.java

Kind: Interface **Lab topic:** LAB 3 – Multiple Inheritance via Interfaces

Purpose: Declares the contract for insured vehicles. Any implementing class must supply getInsuranceInfo() to report the provider and coverage level. Together with GPS, this pair of interfaces is the project's primary demonstration of multiple interface inheritance.

Contract Method

Method	Parameters	Returns	Description
getInsuranceInfo()	—	String	Returns insurance provider name and coverage description

Relationships

Class / File	Relationship	Target	Details
Insured	is implemented by	PremiumVehicle	PremiumVehicle provides the concrete getInsuranceInfo() body

8. InvalidDaysException.java

Kind: Custom Exception (extends Exception) **Lab topic:** LAB 5 – Exception Handling

Purpose: A checked, domain-specific exception thrown whenever the user enters a rental duration of zero or fewer days. Extending java.lang.Exception (rather than RuntimeException) forces all callers to handle or declare it, making the business rule explicit in the method signature.

Constructor

Method	Parameters	Returns	Description
InvalidDaysException(message)	String message	—	Passes the descriptive message to Exception via super(message)

Relationships

Class / File	Relationship	Target	Details
InvalidDaysException	extends	java.lang.Exception	Inherits getMessage(), standard checked-exception behaviour
VehicleRentalGUI	throws	InvalidDaysException	Thrown in calculateRent() when days <= 0; caught in the same

9. DBConnection.java

Kind: Utility Class (static factory) *Lab topic:* LAB 8 – JDBC

Purpose: A thin wrapper around JDBC's DriverManager that centralises the SQLite connection string and driver-registration logic. All other classes that need a Connection call *DBConnection.getConnection()* instead of duplicating the URL.

Fields

Field / Member	Type	Description
DB_URL	private static final String	"jdbc:sqlite:vehicle_rental.db" — path to the SQLite database file

Methods

Method	Parameters	Returns	Description
getConnection()	—	java.sql.Connection	Loads org.sqlite.JDBC driver via Class.forName, then returns D

Relationships

Class / File	Relationship	Target	Details
DBConnection	is used by	DatabaseManager	Every DatabaseManager method calls DBConnection.getConn

10. DatabaseManager.java

Kind: Data-Access Class (static methods) **Lab topic:** LAB 8 – JDBC

Purpose: Provides all CRUD operations on the *rental_history* SQLite table. Uses PreparedStatement for parameterised queries (prevents SQL injection). Resources are always released in finally blocks via the private *closeQuietly* helper.

Database Schema

```
CREATE TABLE IF NOT EXISTS rental_history ( id INTEGER PRIMARY KEY AUTOINCREMENT,
customer_name TEXT NOT NULL, vehicle_type TEXT NOT NULL, days INTEGER NOT NULL, total_cost
REAL NOT NULL, rental_date TEXT NOT NULL )
```

Methods

Method	Parameters	Returns	Description
initializeDatabase()	—	void	Creates rental_history table on first run. Called from Main.main
insertRental(name, type, days, cost, date)	String, String, int, double, String	boolean	Inserts one rental record using a PreparedStatement. Returns
getAllRentals()	—	String	Selects all rows ordered by id DESC; formats them into a print
closeQuietly(resource)	AutoCloseable	void	Private helper — silently closes any AutoCloseable (Connection

Relationships

Class / File	Relationship	Target	Details
DatabaseManager	depends on	DBConnection	Calls DBConnection.getConnection() at the start of each metho
VehicleRentalGUI	calls	DatabaseManager	saveRental() → insertRental(); viewHistory() → getAllRentals()
Main	calls	DatabaseManager	Calls initializeDatabase() once at startup

11. ReceiptGenerator.java

Kind: Utility Class (static methods, File I/O) **Lab topic:** LAB 8 – File I/O

Purpose: Writes a formatted rental receipt to *receipt.txt* in the project root. Uses a try-with-resources block to guarantee that `FileWriter` and `BufferedWriter` are closed even when an `IOException` is thrown.

Fields

Field / Member	Type	Description
RECEIPT_FILE	private static final String	"receipt.txt" — output file name

Methods

Method	Parameters	Returns	Description
generateReceipt(name, type, days, cost, date, double, String)	String, String, String, double, String	boolean	Writes receipt content to RECEIPT_FILE using <code>BufferedWriter</code>
getReceiptFileName()	—	String	Returns the constant file name so callers can display it in dialog

Relationships

Class / File	Relationship	Target	Details
ReceiptGenerator	writes to	receipt.txt (filesystem)	Output file created/overwritten each time a receipt is generated
VehicleRentalGUI	calls	ReceiptGenerator	<code>printReceipt()</code> calls <code>generateReceipt()</code> then shows a dialog with

12. SystemClockThread.java

Kind: Runnable (Background Thread) **Lab topic:** LAB 4 – Multithreading

Purpose: Keeps a live digital clock ticking in the GUI header without blocking the Swing Event Dispatch Thread (EDT). Implements Runnable so the thread logic is decoupled from Thread itself. Uses *volatile boolean running* for a safe cross-thread stop signal and SwingUtilities.invokeLater to push UI updates onto the EDT.

Fields

Field / Member	Type	Description
clockLabel	private final JLabel	Reference to the JLabel in the GUI that displays the current date/time
running	private volatile boolean	Controls the loop; volatile ensures the EDT's stopClock() write is immediately visible

Methods

Method	Parameters	Returns	Description
SystemClockThread(clockLabel)	JLabel	—	Constructor — stores the label reference
run()	—	void	Loop: format current time → invokeLater to update label → Thread.sleep()
stopClock()	—	void	Sets running = false; the loop exits on the next iteration

Relationships

Class / File	Relationship	Target	Details
SystemClockThread	implements	java.lang.Runnable	run() is the thread body
SystemClockThread	updates	JLabel (in VehicleRentalGUI)	Pushes new time string via SwingUtilities.invokeLater
VehicleRentalGUI	creates & owns	SystemClockThread	startClockThread() wraps it in a daemon Thread; dispose() calls stopClock()

13. VehicleRentalGUI.java

Kind: Swing GUI Class (extends JFrame) **Lab topic:** LAB 1 – Method Overloading | LAB 2 – Polymorphism | LAB 5 – Exception Handling | LAB 7 – Swing GUI

Purpose: The central user-facing component. Builds a 750 × 680 px window with a professional soft-blue theme using BorderLayout for the outer structure and GridBagLayout for each section panel. It ties every other class in the project together: it creates Vehicle objects (polymorphism), throws and catches InvalidDaysException, delegates persistence to DatabaseManager, delegates file output to ReceiptGenerator, and manages the SystemClockThread lifecycle.

Key Fields

Field / Member	Type	Description
selectedVehicle	Vehicle (polymorphic)	Holds the active Car / Bike / PremiumVehicle instance
customerNameField	TextField	Customer name input
vehicleComboBox	JComboBox	Car / Bike / Premium Car selector
daysField	TextField	Rental duration input
discountField	TextField	Optional discount % input
resultArea	TextArea	Scrollable output area
clockLabel	JLabel	Live date/time display, updated by SystemClockThread
systemClock	SystemClockThread	Runnable instance passed to the background Thread
lastCalculatedTotal	double	Cached result used by Save and Print Receipt

Key Methods

Method	Parameters	Returns	Description
VehicleRentalGUI()	—	—	Constructor — calls initializeFrame(), buildGUI(), startClockThread()
buildGUI()	—	void	Assembles header, form, and result panels using BorderLayout
createVehicle(type)	String	Vehicle	Factory method — returns Car, Bike, or PremiumVehicle based on type
calculateTotal(v, days)	Vehicle, int	double	Overloaded v1 — no discount; delegates to vehicle.calculateBaseRent()
calculateTotal(v, days, disc%)	Vehicle, int, double	double	Overloaded v2 — applies discount percentage to base rent
calculateRent()	—	void	Button handler — validates inputs, throws InvalidDaysException if invalid
saveRental()	—	void	Calls DatabaseManager.insertRental() with cached rental data
viewHistory()	—	void	Calls DatabaseManager.getAllRentals() and displays result in resultArea
printReceipt()	—	void	Calls ReceiptGenerator.generateReceipt() and shows success message
clearForm()	—	void	Resets all input fields and cached state
startClockThread()	—	void	Creates SystemClockThread as a daemon thread and starts it
dispose()	—	void	Overrides JFrame.dispose() to call systemClock.stopClock() before disposing

Relationships

Class / File	Relationship	Target	Details
VehicleRentalGUI	extends	javafx.swing.JFrame	Is a top-level window; EXIT_ON_CLOSE terminates the JVM
VehicleRentalGUI	owns (polymorphic)	Vehicle / Car / Bike / PremiumVehicle	Vehicle declared as Vehicle; actual type chosen at runtime
VehicleRentalGUI	catches	InvalidDaysException	calculateRent() throws and immediately catches it in the same method
VehicleRentalGUI	calls	DatabaseManager	saveRental() and viewHistory() delegate to static DatabaseManager
VehicleRentalGUI	calls	ReceiptGenerator	printReceipt() calls ReceiptGenerator.generateReceipt()
VehicleRentalGUI	creates & manages	SystemClockThread	startClockThread(); stopClock() called in dispose()

14. Main.java

Kind: Entry-Point Class **Lab topic:** Application Bootstrap

Purpose: The single class containing *public static void main*. Its only responsibilities are: (1) call `DatabaseManager.initializeDatabase()` to ensure the schema exists before any GUI is shown, and (2) schedule the creation of `VehicleRentalGUI` on Swing's Event Dispatch Thread using `SwingUtilities.invokeLater`.

Execution Flow

```
main(String[] args) ■ ■■ DatabaseManager.initializeDatabase() // creates table if absent ■  
■■■ SwingUtilities.invokeLater(...) ■ ■■ UIManager.setLookAndFeel(system) // optional native  
theme ■ ■■ new VehicleRentalGUI().setVisible(true) ■■ (all GUI + thread lifecycle managed  
inside VehicleRentalGUI)
```

Relationships

Class / File	Relationship	Target	Details
Main	calls	DatabaseManager	<code>initializeDatabase()</code> before GUI launches
Main	instantiates	VehicleRentalGUI	Creates and shows the main window on the EDT

15. Complete Class Relationship Summary

Inheritance & Interface Relationships

Class / File	Relationship	Target	Details
Car	extends	Vehicle	Single class inheritance
Bike	extends	Vehicle	Single class inheritance
PremiumVehicle	extends	Car	Single class inheritance (chained)
PremiumVehicle	implements	Insured	Interface contract: getInsuranceInfo()
PremiumVehicle	implements	GPS	Interface contract: getGPSInfo()
InvalidDaysException	extends	Exception	Custom checked exception
VehicleRentalGUI	extends	JFrame	Top-level Swing window
SystemClockThread	implements	Runnable	Thread body pattern

Usage / Dependency Relationships

Class / File	Relationship	Target	Details
Main	calls	DatabaseManager	initializeDatabase() at startup
Main	instantiates	VehicleRentalGUI	Creates the window on the EDT
VehicleRentalGUI	polymorphically uses	Vehicle hierarchy	selectedVehicle field holds Car / Bike / PremiumVehicle at runtime
VehicleRentalGUI	throws / catches	InvalidDaysException	Days validation in calculateRent()
VehicleRentalGUI	calls	DatabaseManager	insertRental(), getAllRentals()
VehicleRentalGUI	calls	ReceiptGenerator	generateReceipt(), getReceiptFileName()
VehicleRentalGUI	creates & manages	SystemClockThread	Background daemon thread for live clock
DatabaseManager	calls	DBConnection	getConnection() for every DB operation
SystemClockThread	updates (EDT)	JLabel (in GUI)	Via SwingUtilities.invokeLater

OOP Concept Mapping

OOP Concept	Where Demonstrated	Lab
Abstraction	Vehicle (abstract class + abstract methods)	LAB 6
Encapsulation	private/protected fields + getters in all domain classes	LAB 1
Inheritance	Car, Bike extend Vehicle; PremiumVehicle extends Car	LAB 2
Polymorphism	Vehicle selectedVehicle in GUI; calculateBaseRent() dispatched at runtime	LAB 2
Method Overloading	Two calculateTotal() signatures in VehicleRentalGUI	LAB 1
Multiple Interfaces	PremiumVehicle implements Insured + GPS	LAB 3
Custom Exception	InvalidDaysException thrown/caught in calculateRent()	LAB 5

Multithreading	SystemClockThread runs in daemon Thread; volatile flag for stop	LAB 4
Swing GUI	VehicleRentalGUI (JFrame, JPanel, JButton, JComboBox, etc.)	LAB 7
JDBC	DBConnection + DatabaseManager: PreparedStatement, ResultSet	LAB 8
File I/O	ReceiptGenerator: FileWriter + BufferedWriter + try-with-resources	LAB 8